

full-time job and is the reason so many hobby scrapers quietly stop working.

The second is JavaScript-rendered content. A growing share of pages ship almost no data in the initial HTML. What you get is a skeleton, and a script that fetches the real content and paints it into the page in your browser. Fetch that page with a plain HTTP client and the fields you want are simply not there. To see them you need to run an actual browser engine, wait for the right network calls to finish, and only then read the DOM.

The third is brittle extraction. Even when you can fetch a page, you still have to pull the fields out of it, and the usual approach is a stack of CSS or XPath selectors tuned to one site's markup. Those selectors are fragile by nature. A layout change, an A/B test, or a seasonal redesign can silently break them, and you often find out only when your data goes blank or wrong. Multiply that by every site you scrape and maintenance becomes the main cost of the whole operation.

The rest of this is about taking down each of those walls with as little code as possible.

One API call, three hard problems

Instead of assembling a proxy pool, a browser farm, and a pile of per-site parsers yourself, you send one HTTP request to a single endpoint and ask for exactly the level of help you need. The Zyte API works this way. Every request is a POST to `https://api.zyte.com/v1/extract`, authenticated with HTTP basic auth where your API key is the user name and the password is empty. What you get back depends entirely on which fields you set in the JSON body (Figure 1).

Start with the simplest case: you just want the HTML, but the site keeps blocking you. You ask for the raw response body and let the API handle the unblocking.

```
curl --user YOUR_ZYTE_API_KEY: \
  --header 'Content-Type: application/json' \
  --data '{"url": "https://books.toscrape.com", "httpResponseBody": true}' \
  --compressed \
  https://api.zyte.com/v1/extract
```

The response is JSON, and `httpResponseBody` comes back Base64-encoded, so you decode it to get the HTML. When I ran this against the sandbox site above, the decoded body was 51,274 bytes of ordinary markup, fetched without me running a single proxy. Unblocking is no longer infrastructure you operate; it is just a field you set to true.

When the page needs a real browser, you flip a different field. Setting `browserHtml` to true tells the API to load the page in a headless browser, run its JavaScript, and return the rendered DOM instead of the raw source. The same request can carry a list of actions to click, scroll, or wait for a selector before capturing, take a screenshot, or set geolocation to fetch the page as if from a specific country. You are still sending one request to one endpoint; you are just describing more of what a browser would do.

The part that changes how you build scrapers is automatic extraction. Instead of asking for HTML at all, you ask for a structured data type and let the API's extraction models do the parsing. For a

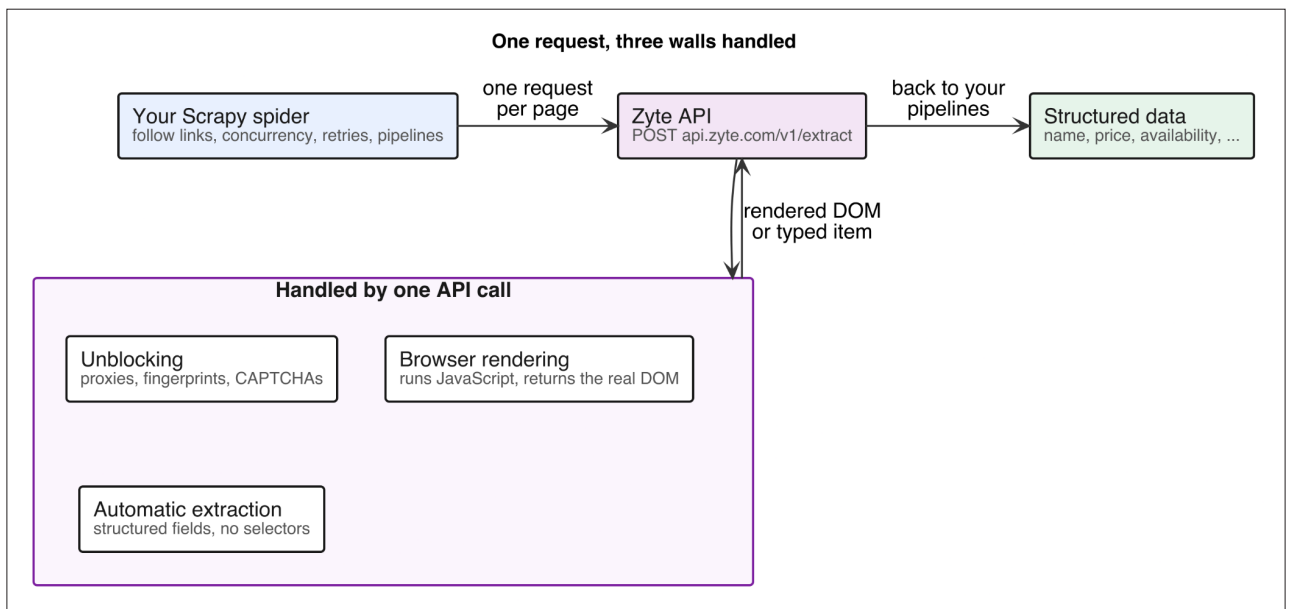


Figure 1: One request, three walls handled. Scrapy drives the crawl while a single API call takes care of unblocking, browser rendering, and automatic extraction, then hands structured data back to your pipelines